

Projeto Sonora - Fase 02: Tratamento de exceções

O **Sonora**, funciona, compila, faz o cadastro rodar. Mas ele é ingênuo: se você mandar criar uma música com duração negativa, ele obedece. Se o usuário digitar letra onde era para digitar número, o programa explode na cara dele com aquela pilha de vermelho no console. Nesta fase a gente resolve isso.

O combinado continua o mesmo de sempre:

1. Você clona a pasta da fase anterior para uma nova (o sonora-fase01 vira sonora-fase02).
2. A Fase 01 fica congelada, intacta. Você não mexe mais nela.
3. Você evolui as **mesmas classes** aplicando o que aprendeu sobre exceções, sem quebrar o que já funcionava.

Ou seja: se a sua Fase 01 estava bem feita, esta fase vai fluir. Se estava torta, agora é a hora de endireitar. Nome da pasta nova: sonora-fase02.

O foco desta fase é deixar o sistema robusto: impedir que objetos nasçam em estado inválido, transformar parte da sinalização por valor de retorno em lançamento de exceção, e blindar o menu contra digitação errada. Nada de coleção, nada de herança ainda; esses assuntos continuam guardados para as próximas unidades.

2. O que você PODE e o que você NÃO PODE usar nesta fase

Pode usar (e vai precisar):

- Tudo que você já usava na Fase 01 (classes, private, construtores, static, sobrecarga, arrays, Scanner).
- throw para lançar exceções.
- As exceções prontas da própria linguagem: IllegalArgumentException, IllegalStateException, NumberFormatException, ArithmeticException, ArrayIndexOutOfBoundsException e afins.
- try, catch (inclusive múltiplos catch e catch por superclasse) e finally.

NÃO pode usar nesta fase:

- ArrayList, List, Map, Set ou qualquer coleção do java.util para guardar dados (continua tudo em array).
- Herança (extends) ou interfaces (implements). Isso ainda não foi visto.
- Criar suas próprias classes de exceção (subclasses de Exception). Nesta fase você usa só as exceções prontas da linguagem.
- Leitura ou escrita de arquivos e bibliotecas externas.

Sobre a cláusula throws: os slides mostram que ela serve para delegar ao método chamador o tratamento de exceções *verificadas* (as que o compilador obriga a tratar, como as de arquivo). Como nesta fase você só vai lançar exceções *não verificadas* (as subclasses de RuntimeException, como IllegalArgumentException), você **não precisa** escrever throws nas assinaturas. É bom saber que ela existe e para que serve; usá-la aqui é opcional.

3. Blindando os construtores e setters (lançamento de exceção)

Na Fase 01, um objeto podia nascer zoadado e ninguém reclamava. Agora não. A regra de ouro desta seção é: **um objeto não pode existir em estado inválido**. Se os dados que chegam no construtor não fazem sentido, o construtor lança uma exceção e o objeto simplesmente não nasce.

O padrão é o mesmo do exemplo do `setSalario` que vimos em aula: valida primeiro, lança se estiver errado, e só então atribui.

```
public void setSalario(double novoSalario) {  
    if (novoSalario < 0) {  
        throw new IllegalArgumentException("Salário incorreto");  
    }  
    salario = novoSalario;  
}
```

Aplique essa mesma ideia no `Sonora`. As validações obrigatórias são estas:

3.1. Classe Musica

- `titulo` não pode ser `null` nem vazio (nem só espaços). Se for, lance `IllegalArgumentException` com uma mensagem clara.
- `artista` não pode ser `null` nem vazio, mesma regra.
- `duracaoSegundos` precisa ser maior que zero. Música de 0 segundo ou de duração negativa não existe.
- Essas validações valem no construtor. Como não há setter público para esses campos, o construtor é o único portão de entrada, e é ele que tem que barrar.

3.2. Classe Usuario

- `nome` não pode ser `null` nem vazio.
- `email` não pode ser `null` nem vazio, e precisa conter um `@`. Não precisa validar e-mail de verdade (isso é um mundo à parte); basta checar que não está vazio e que tem um `@` no meio.

3.3. Classe Playlist

- `nome` não pode ser `null` nem vazio.
- `dono` não pode ser `null`. Playlist sem dono não faz sentido no nosso modelo.

Detalhe importante sobre a mensagem. A mensagem da exceção não é decoração: ela é a informação que descreve o erro (como visto em aula, o objeto de exceção carrega dados sobre o que deu errado). Escreva mensagens que ajudem a entender o problema. "Duração inválida: -30. A duração deve ser maior que zero." é muito melhor que um seco "erro".

4. Trocando sinalização por retorno por lançamento de exceção

Na Fase 01, vários métodos avisavam que algo deu errado devolvendo um valor: adicionar devolvia `false`, as buscas devolviam `null`. Aquilo funcionava, mas nem todo problema tem o mesmo peso. Parte dessas situações vira exceção agora, parte continua como retorno. E entender **o que vira o quê** é justamente o aprendizado desta seção.

4.1. O que VIRA exceção

Situações que representam **uso indevido** da classe (o programador chamou o método com um argumento que nunca deveria ter passado) merecem exceção. Ajuste a classe `Playlist`:

- `getNaPosicao(int indice)`: se o índice estiver fora do intervalo válido, em vez de devolver `null`, lance `IndexOutOfBoundsException`. Pedir a posição 50 de uma playlist que tem 3 músicas é um erro de quem chamou, não uma resposta normal.
- `removerNaPosicao(int indice)`: mesma coisa. Índice inválido agora lança `IndexOutOfBoundsException` em vez de devolver `false`.

4.2. O que CONTINUA como retorno

Nem tudo vira exceção. Situações que são um resultado **esperado e legítimo** da operação continuam sinalizadas por retorno:

- `adicionar(Musica musica)` na `Playlist`: adicionar numa playlist cheia continua devolvendo `false`. `Playlist` cheia é um estado normal e previsível do sistema, não um erro de programação. (Já passar uma música `null` para adicionar é uso indevido: esse caso vira `IllegalArgumentException`.)
- `buscarMusicaPorId(int)` e `buscarMusica(String)` na `Plataforma`: não achar continua devolvendo `null`. Procurar e não encontrar é uma resposta válida de uma busca, não uma anomalia.

A regra prática: se é o programador usando a classe errado, é exceção. Se é uma resposta possível e prevista do fluxo normal, é retorno. Guarde essa distinção; ela vale para muito além do Sonora.

5. Blindando o menu do App (try/catch, múltiplos catch e finally)

Aqui é onde o usuário digita coisa errada e o programa **não pode** morrer por causa disso. Na Fase 01 a dica era usar `hasNextInt()` como paliativo. Agora você faz do jeito certo, com tratamento de exceção.

5.1. Leitura de opções e de números

Leia a entrada do usuário como texto e converta com `Integer.parseInt` dentro de um `try`. Se vier "abc" onde era para vir um número, o `parseInt` lança `NumberFormatException`, e o seu `catch` avisa o usuário e deixa ele tentar de novo, sem derrubar o programa. É o mesmo padrão do laço que vimos em aula:

```
Scanner teclado = new Scanner(System.in);
int opcao;
while (true) {
    try {
        System.out.print("Escolha uma opcao: ");
        opcao = Integer.parseInt(teclado.nextLine());
        break;
    } catch (NumberFormatException e) {
        System.out.println("Valor invalido. Digite um numero.");
    }
}
```

5.2. Capturando as exceções que as suas classes agora lançam

Como os construtores e alguns métodos passaram a lançar exceção, as chamadas a eles dentro do menu precisam estar protegidas. Quando o usuário cadastra uma música com duração negativa, o `catch` deve

mostrar a mensagem da exceção (e não deixar o programa cair). Uma forma limpa de mostrar a mensagem é usar `e.getMessage()`:

```
try {
    Musica m = new Musica(titulo, artista, duracao);
    plataforma.cadastrarMusica(m);
    System.out.println("Musica cadastrada!");
} catch (IllegalArgumentException e) {
    System.out.println("Nao foi possivel cadastrar: " + e.getMessage());
}
```

5.3. Pelo menos um bloco com múltiplos catch

Em pelo menos um ponto do App (a operação de reproduzir ou remover por posição é uma boa candidata), monte um try com **mais de um catch**, tratando exceções diferentes de formas diferentes. Lembre da regra que vimos em aula: as cláusulas catch são testadas **na ordem**, então uma exceção mais específica (`NumberFormatException`) vem **antes** de uma mais genérica (`Exception`). Colocar a superclasse primeiro é erro de compilação.

```
try {
    int pos = Integer.parseInt(teclado.nextLine());
    Musica m = playlist.getNaPosicao(pos);
    m.reproduzir();
    System.out.println("Tocando: " + m.getTitulo());
} catch (NumberFormatException e) {
    System.out.println("A posicao precisa ser um numero.");
} catch (IndexOutOfBoundsException e) {
    System.out.println("Essa posicao nao existe na playlist.");
}
```

5.4. Um finally para fechar

Use `finally` em pelo menos um try do seu App, para praticar o conceito. Como vimos, o `finally` roda **sempre**, dando erro ou não, e costuma ser usado para liberar recursos. Um uso honesto aqui é imprimir uma linha de separação ou uma mensagem de "operacao finalizada" que deve aparecer independentemente de ter dado certo ou não. Não force um `finally` inútil em todo lugar; um bem colocado já mostra que você entendeu para que serve.

6. Comportamentos que precisam funcionar (e onde está a cobrança)

De novo, o que separa um "compilou" de um trabalho bem feito. Na correção eu vou testar cada um destes:

- **Objeto não nasce inválido.** Tentar criar `new Musica("", "X", 100)` ou `new Musica("Y", "Z", -5)` lança `IllegalArgumentException`. O objeto não é criado.
- **A mensagem descreve o erro.** A exceção carrega uma mensagem que faz sentido, não um "erro" genérico.
- **Exceção onde é uso indevido.** `getNaPosicao` e `removerNaPosicao` com índice fora do intervalo lançam `IndexOutOfBoundsException`.

- **Retorno onde é fluxo normal.** adicionar numa playlist cheia ainda devolve false; buscar algo inexistente ainda devolve null.
- **O menu não morre.** Digitar texto onde se espera número não derruba o programa: o catch avisa e o menu segue rodando.
- **Múltiplos catch na ordem certa.** Existe pelo menos um try com mais de um catch, do mais específico para o mais genérico.
- **Um finally presente e com propósito.** Existe pelo menos um finally, e ele faz algo que se justifica.

7. Roteiro de demonstração (você tem que conseguir reproduzir tudo isto)

Prepare seu App para, numa demonstração, mostrar cada um destes comportamentos. Na correção eu sigo mais ou menos esta lista:

1. Tentar cadastrar uma música com título vazio e mostrar que a `IllegalArgumentException` foi lançada e tratada (o programa avisa e continua vivo).
2. Tentar cadastrar uma música com duração zero ou negativa e mostrar o mesmo comportamento.
3. Tentar cadastrar um usuário com e-mail sem @ e mostrar a exceção sendo tratada.
4. Pedir `getNaPosicao` de uma posição que não existe e mostrar que agora vem `IndexOutOfBoundsException` (tratada), e não null.
5. Encher a playlist e tentar adicionar a 101a, mostrando que adicionar ainda devolve false (isto continua sendo retorno, não exceção).
6. Buscar uma música por um id que não existe e mostrar que ainda vem null (também continua retorno).
7. No menu, digitar uma letra onde se espera um número e mostrar que o programa avisa e pede de novo, sem cair.
8. Disparar um bloco com múltiplos catch e provocar duas exceções diferentes para mostrar cada catch agindo.
9. Mostrar o finally executando tanto num caso de sucesso quanto num caso de erro.

Sugestão forte: coloque esses passos como um roteiro no seu README, igual você fez (ou deveria ter feito) na Fase 01.

8. Erros comuns que vão te dar dor de cabeça (evite)

- Validar no construtor mas **atribuir antes** de validar. Valide primeiro, atribua depois; senão o objeto chega a segurar o valor inválido por um instante.
- Trocar **tudo** por exceção, inclusive a playlist cheia e a busca sem resultado. Rer a seção 4: isso é fluxo normal, continua sendo retorno.
- Colocar catch (`Exception` e) **antes** de um catch mais específico. Isso nem compila: a superclasse tem que vir por último.
- Engolir a exceção com um catch vazio. Um catch que não faz nada esconde o erro e é pior que não ter catch. No mínimo, mostre a mensagem.



- Deixar o construtor lançar exceção mas **não proteger** a chamada no menu. Aí o objeto não nasce, mas o programa cai do mesmo jeito, porque a exceção subiu até o main sem ninguém tratar (propagação, lembra?).
- Escrever throws achando que resolve. throws só delega o tratamento para quem chamou; alguém, em algum ponto da cadeia, ainda precisa **tratar** com try/catch.
- Mensagem de exceção inútil ("erro", "deu ruim"). A mensagem é informação, use-a.